PROJECT ASSIGNMENT GUIDELINES

João Rafael Duarte Almeida

v19.02.26

Index

1 Project objectives 2
1.1 Project assignment objectives 2
1.2 Team and roles 2
1.3 Product concept 3
1.3.1 Generic product requirements 3
1.3.2 Architectural requirements 4
1.3.3 Extra credits (advanced topics) 6
2 Project implementation 6
2.1 Required practices 6
2.1.1 Agile *backlog* management (plan & track) 6
2.1.2 Feature-branching workflow 6
2.1.3 Containers-based deployment 7
2.2 Project outcomes/artifacts 7
2.2.1 Project repository (Git) 7
2.2.2 Project requirements and technical specifications 8
2.2.3 API documentation 8
2.3 Project schedule: iteration plan 8
2.3.1 Deliverables 9
2.3.2 Meeting minutes 9
2.3.3 Expected results 9
2.3.4 Sprints guidelines 10

1 Project objectives

1.1 Project assignment objectives

The project assignment will require that students apply selected software engineering practices, including:

- Develop a product specification, from the usage scenarios (user stories) to the technical design.
- Propose, justify, and implement a software architecture, based on enterprise *frameworks*.
- Apply collaborative work practices, both in code development and agile project management.

1.2 Team and roles

Each team/group should assign the roles described in Table 1. Note that the team should perform the activities collaboratively; the idea of *roles* is to have people in the team to lead a specific part of the work and act as a spokesperson for that specific area.

Table 1: Selected roles for the software development team.

Role	**Key responsibilities**
Team manager (coordinator)	Actively promote the best collaboration in the team and take the initiative to address problems that may arise. Ensure the necessary discussion so there is a fair distribution of tasks and that members work according to the plan. Ensure that the requested project outcomes are delivered in time.
Product owner	Represents the interests of the stakeholders. Has a deep understand of the product and the application domain; the team will turn into the Product Owner to clarify questions about product features/requirements. Responsible for accepting the solution increments.
Architect	Deep understanding of the proposed architecture and supporting technologies. The team will turn into the Architect to explain the expected behavior of each software component and interactions between modules.
DevOps master	Responsible for the infrastructure and its configuration; ensures that the development framework works properly. In-charge of preparing the deployment machine(s)/containers, Git repository, cloud infrastructure, databases operations, etc.
Developer	ALL members contribute to the development tasks.

1.3 Product concept

Each group is expected to propose, conceptualize, and implement a multi-layer, enterprise-class solution observing the guidelines in the following sections.

1.3.1 Generic product requirements

The scope of the proposed solution should include:

- distributed (remote) **generation of data streams**, either from real data sources (e.g.: collecting data from devices) or by virtual software "agents" (e.g.: simulating the behavior of a Stock market)
- **data publishing** using a lightweight, message-oriented protocol (push data upward: remote → central backend).
- long term **storage** of data using a persistent database engine.
- **central processing**, ensuring the detection of relevant events (e.g.: alarms) and data aggregation.
- **service API** (REST) providing a comprehensive set of endpoints to access data and manage the system.
- a **web portal** to implement the core user stories. Using the web portal, the users will be able to track the current updates (access the upstream data) and query/filter stored data (historical views). In addition, you can provide a **mobile application** as an alternative presentation frontend.

The teams must select one of the following applications and confirm its availability with the instructors. The available topics are listed below:

1. A web platform that consumes soil moisture, temperature, and weather station sensor data to optimize irrigation and fertilization in precision agriculture.
2. A health diary web application that aggregates heart rate, sleep, step count, and SpO_2 sensor data from wearables to monitor long-term well-being.
3. A smart home dashboard that processes temperature, motion, light, and energy consumption sensor data to automate and optimize domestic environments.
4. A smart city platform that analyzes air quality, traffic density, and noise level sensor data to support urban planning and sustainability decisions.
5. A food delivery monitoring system that consumes GPS and temperature sensor data to track delivery performance and cold-chain compliance.
6. A smart parking web application that integrates occupancy and vehicle detection sensor data to optimize parking space allocation.
7. A livestock monitoring platform that processes body temperature, movement, and location sensor data to detect health anomalies in farm animals.
8. An industrial monitoring system that analyzes vibration, pressure, and temperature sensor data to enable predictive maintenance of machinery.
9. A smart campus platform that consumes occupancy, CO_2, humidity, and energy sensor data to improve comfort and resource efficiency.

10. A flood early-warning web system that integrates rainfall, river level, and soil saturation sensor data to predict and visualize risk scenarios.
11. A remote patient monitoring application that processes ECG, glucose, blood pressure, and motion sensor data for continuous healthcare supervision.
12. A smart waste management system that consumes bin fill-level and GPS sensor data to optimize collection routes and schedules.
13. A workplace safety platform that analyzes gas detection, noise level, and environmental sensor data to ensure regulatory compliance.
14. A sports performance web application that integrates accelerometer, gyroscope, and heart rate sensor data to optimize athlete training.
15. A structural health monitoring platform that processes vibration, strain, and tilt sensor data to assess building and infrastructure integrity.

After choosing the topic, students must clearly define the intended scope:

Sample project theme: a wind farms management system.

- **Concept**: an integrated platform to monitor and control remote wind turbines, in wind farms, distributed by multiple sites.
- **Data acquisition layer**: wind turbines telemetry (rpm, energy generation output,...) is being continuous collected; the environment parameters (wind speed, wind orientation,…) are also tracked.
- **Data publishing**: at each wind farm, there is site gateway, acting as a local area aggregator and sends the telemetry data to the cloud, using a bandwidth-savvy protocol.
- **Processing & bizz logic**: hazards conditions are detected when the telemetry reveals that operating thresholds are compromised. In this case, the alarms are also forwarded to the mobile devices of the people in charge. Peak conditions are detected in an hourly basis (data aggregation) to build historic trends.
- **Integration API**: the computational platform exposes endpoints that allows other authorized applications to programmatically list the park information system (e.g.: farm location, details of each tower,…) and telemetry readings, both current and for past intervals of interest.
- **Web portal**: the web application allows basic tracking of operational conditions (telemetry dashboard of the turbines); browse descriptive information on each tower/device (model, location,...); adjust operational parameters, actuating in the wind farm (e.g.: stop a turbine).

1.3.2 Architectural requirements

The solution should adopt, in general, a multilayer architecture as depicted (Figure 1).

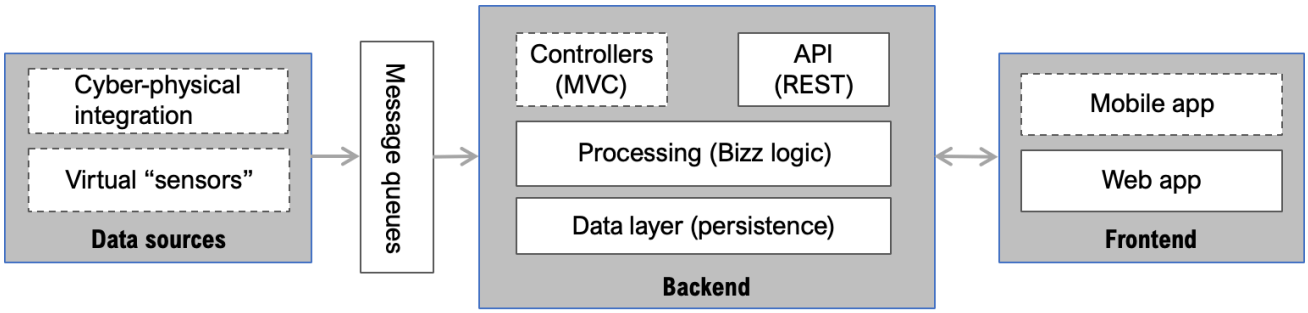


Figure 1: High-level reference architecture (should be adapted and detailed for each project; dashed components may be relevant or not, depending on the approach).

Notes on the expected components and implementation technologies:

a) **Data sources:** the system should ingest data streams, typically generated at cyber-physical systems (e.g.: environmental sensors, cameras, location tracking, devices telemetry, body sensors, etc.) or by virtual software "sensors". The use of hardware devices is optional; you may use "digital twins" instead (virtual representation of a device, simulated in software); or event "agents" that generate data streams that are not related to hardware (e.g.: outcome lab tests for COVID-19).

b) **Message queues**. Data ingestion (from streams into the backend) should use a lightweight message-oriented middleware.

c) **API (REST)**. Endpoints to allow presentation layers (or other systems) to connect to your own services. The API should follow the REST style, with JSON payloads. Note that a comprehensive API would allow you to access (almost) all system services through the API.

d) **Processing & bizz logic**. This module will likely be divided into several sub-modules, depending on the specific problem domain. You should have some processing module to analyze the incoming streams (likely to integrate with the message broker).

e) **Data access**. Data from streams, and the information system to support other parts of the system (e.g.: users, profiles, preferences), is to be persisted using an object-database mapping framework, like Spring Data. The system will save the data into a convenient database technology. While the Relational databases are obvious options (e.g.: Postgresql, MySql), you may use other solutions that better fit the project requirements (e.g.: time-based queries will benefit from a timeseries repository).

f) **Web app**: you may choose the web development framework and integrate with the backend using the services API. If you choose to use a Java-related technology (e.g.: Thymeleaf and Spring MVC), the web layer may use the controllers in the backend (bypassing the API). If you choose a JavaScript-based framework, for example, the presentation layer would interact with the API.

g) **Mobile app** (optional): a simple application that interacts with the backend using the REST API. Push notifications would be an additional challenge.

Additional notes:

h) You need to implement at least a presentation platform (web app).

i) There could be other elements, depending on the problem. E.g.: you are not asked to implement actuation (send control instructions from the backend to the devices) but it may be important for your scenario.

1.3.3 Extra credits (advanced topics)

The following challenges are not mandatory, but may give you extra credits in the project, depending on the quality of the implementation:

- Robust data analytics (integrate a framework for streams processing at the backend)
- Instrumentation to monitor the production environment (e.g.: *ELK stack* for application logs analysis)
- Use of relevant cyber physical components (e.g.: sensors or actuators connected to a RPi board) to deliver value to the end-users.

2 Project implementation

2.1 Required practices

2.1.1 Agile *backlog* management (plan & track)

The team will use a backlog to prioritize, assign and track the development work.

This backlog follows the principles of "agile methods" and use the concept of "user story" as the unit for planning. Stories are briefly documented declaring the benefit that a given *persona* wants to get from the system.

Stories have points, which "quantifies" the shared expectation about the effort the team plans for the story, and prioritized, at least, for the current iteration. Developers start work on the stories on the top of the current iteration queue, adopting an agreed workflow.

The backlog management should be done using GitHub Projects + Scrum boards. The backlog should be consistent with the development activities of the team; both the project management environment and Git repository activity log should provide faithful evidence of the teamwork. You must create the project repository using the following assignment https://classroom.github.com/a/HDdSyidB.

2.1.2 Feature-branching workflow

There are several strategies to manage the shared code repository, and you are required to adopt one in your team. Consider using the "Git Flow".

The **feature-branches should be traceable to user-stories** in the backlog.

Complement this practice by issuing a "pull request" (a.k.a. merge request) strategy to review, discuss and optionally integrate increments in the *main*.

2.1.3 Containers-based deployment

Your logical architecture should apply the principle of responsibilities segregation. Accordingly, the deployment of services should separate the services into specialized containers (e.g.: Docker containers). Your containers will likely map the architecture logic layers. Your solution needs to be deployed into a server environment (e.g.: Cloud infrastructures) using more than one "slice"/container[1].

2.2 Project outcomes/artifacts

2.2.1 Project repository (Git)

The project outcomes should be organized in a cloud-based Git repository. Besides the code itself, teams are expected to include other project outcomes, such as requested documentation. The project must be shared with the faculty staff. Expected repository structure:

readme.md

🗁 reports/

🗁 minutes/

🗁 presentations/

🗁 *projX*/

🗁 *projY*/

Part	**Content:**
README.md	Context and project bookmark placeholder. Be sure to include the following sections: — Project abstract: title and brief description of the project features. — Project team: students' identification and the assigned roles. — Architecture Diagram: updated diagram each iteration — [Project] Bookmarks: links to quickly access all project resources, such as project management boards, editable versions of the reports in the cloud, entry point for your API documentation,....

[1] Although it is likely that your use Docker containers, other options may be considered, depending on the hosting infrastructure.

🗁 reports/	Project specifications, as PDF files.
🗁 minutes/	Meeting minutes in the group (each meeting should result in a minute).
🗁 presentations/	Materials used in project-related presentations.
🗁 *projX*/…	The source code for subproject "projX". The amount and names of subprojects depend on each problem.

2.2.2 Project requirements and technical specifications

The project documentation should be kept in the main branch of the repository ([main]/reports) and updated accordingly. The requirements should be created only on the board as GitHub issues.

The "Project Specification Report" [see sample template on the course page] should cover:

A. Product concept
   A.1. Vision statement
   A.2. Personas
   A.3. Supported scenarios (user stories)
B. Architecture notebook
   B.1. Key quality requirements
   B.2. Architectural view
   B.3. Module interactions [dynamic view]
C. Information model

2.2.3 API documentation

Deliver (and maintain) an autonomous, self-contained report (or an interactive web page) that fully describes the service's API. The documentation should present a clear overview of the API's purpose and structure, detail the available endpoints/methods (including request parameters, response formats, and possible error codes), and outline the expected usage patterns. Where applicable, include examples of typical requests and responses to illustrate correct implementation. Consider using the OpenAPI/Swagger framework to create the API documentation.

2.3 Project schedule: iteration plan

The project will be developed in 2-week iterations. Active management of the product backlog will function as the main source of progress tracking and work assignment.

2.3.1 Deliverables

Each iteration should be associated to a git tag. For instance, iteration I1.1 should belong to tag 0.1.1. The release should have the changelog with the contributors of each feature/issues/pull request associated. Example:

Added

*- **New user interface**: Description (#PR, contributed by First_Last_Name).*

Changed

*- **Enhancement of user interface**: Description (#PR, contributed by First_Last_Name).*

2.3.2 Meeting minutes

Students will meet during classes and outside of class to coordinate the project execution. Each meeting must result in meeting minutes. These minutes should be available in the repository and must be presented in each sprint. The recommended structure is:

- Participants
- Agenda
- Decisions
- Tasks

2.3.3 Expected results

Expected results from project iterations:

Iter. #	**Focus**	**Required outcomes**
I1 **0.1.***	Project initiation (define the concept, setup the tools, prioritize user stories).	• Core stories defined and prioritized (8 – 12 stories). • Backlog management system setup. • Team repository in place.

Iter. #	Focus	Required outcomes
I2 **0.2.***	Define the product architecture. UX preview.	• Draft Project Specification (report); shall include the Architecture Notebook part. • Prototypes[2] for the core user stories (8 – 12 views).
I3 **0.3.***	Stablishing the API. Demonstrate the architecture end-to-end using Postman platform.	• Define and implement the REST API endpoints (CRUD). • Increment deployed (in containers) at the server environment. • Create the documentation for the API (increment report).
I4 **0.4.***	Stabilize the Minimal Viable Product (MVP) using a front-end framework.	• Develop the prototypes using the front-end framework. • Consume the REST API endpoints in the web interfaces. • Increment deployed (in containers) at the server environment. • Update documentation (report).
I5 **1.0.***	Final delivery.	• Basic data pipeline in-place: data streams generation, transmission, and storage. • Additional user stories required for a functional MVP deployed, specially covering data aggregation/ visualization of historic data. • Stabilize the production environment. • Final deployment on the server. • Oral presentation/defense. • Update documentation (final report for project specifications and software documentation).

2.3.4 Sprints guidelines

In each sprint, it is expected that students present, at least, the following documents:

- Meeting minutes since previous sprint;
- Iteration backlog at the current stage;
- Planning of the backlog for the next iteration;
- List of pull request and issues created during the present iteration.

[2] These "prototypes" would be early versions of the web pages, essentially "static" (not yet integrated with the business logic), already implemented with the target technologies. If not possible, then mockups).